ASP.NET MVC Core vs. Web Forms

The first version of ASP.NET was released in 2002, with the Web Forms view engine being the only choice available.

Microsoft had a very commendable goal when they created Web Forms: They wanted to abstract as much of the gritty details of the HTTP protocol and the stateless nature of it away and make web development feel much more like developing a Windows application, which was already a pretty pleasant experience at the time.

They did so by introducing the ViewState, which would make sure that the current state of any form was preserved during post backs to the server and they did it with server controls, which encapsulated the rendering of HTML and CSS into an arbitrary control which you could customize using logical properties instead of being forced to mix HTML and CSS directly.

They also introduced the event driven model, already known to the Windows developers at that time, to allow the developer to respond to actual events, like a button being clicked or a checkbox being changed, instead of doing manual checks for this each time the page was loaded. This also meant that markup and code was separated, which in theory is a great thing.

Where Web Forms Failed

Web Forms was a fresh breath of air for many developers, and it likely also helped a lot of new developers, or developers only familiar with Windows application development, to learn building applications for the web. Unfortunately, Microsoft didn't succeed in creating the perfect and flawless abstraction, because a number of problems quickly emerged. Some of them were fixed in later releases, while others were more fundamental to the way WebForms worked and therefore harder to change. The Web Forms technology has been criticized mainly for the following things:

ViewState makes pages heavier

By keeping track of every server control on the page in a ViewState string, which is sent back and forth between the server on each request, Web Forms pages got quite a bit heavier. If you were building a medium complex page, the resulting ViewState string could lead to an increase of several hundreds of kilobytes. This could lead to longer load times, especially on a mobile connection and with the increase of traffic from smartphones all over the world, this became a very real problem.

Server controls limits your control over HTML output

Server controls makes it easy for you to quickly create something useful, but you never get full control of the HTML which it renders. This can become a problem when you need to fine-tune your work as well as if you experience browser compatibility problems.

Where ASP.NET MVC is an improvement over Web Forms

Microsoft introduced ASP.NET MVC in 2007, with the first stable release being available in 2009. ASP.NET MVC removes a lot of the abstractions implemented by Web Forms. For instance, you usually generate all of the HTML yourself, instead of relying on server controls. There is also no longer any ViewState maintained for you, effectively eliminating that problem (but also rendering several of the server controls, like the GridView and the Repeater, useless at the same time).

Which technology should you choose?

It's important to state that while Web Forms may seem like an outdated technology when reading the above, it's actually not at all - Web Forms is still being actively developed by Microsoft and is still a possible choice when entering the world of web development with ASP.NET. Web Forms is especially well suited for situations where you want something up and running in a hurry - the big amount of advanced server controls makes it easy to accomplish something very useful in a rush, at the price of the flexibility it gives you to write all the markup manually.

If you already know how to use Web Forms, you should definitely give ASP.NET MVC a try, especially if some of the above mentioned problems have been bugging you as well.

QUICK VIEW OF ASP.NET MVC

ASP.NET is very popular framework for developing Web Based Application. ASP.NET MVC is based on Model – View – Controller pattern that makes it more robust, fast, and secure. ASP.NET MVC is the next generation technology and it is the successor of ASP.NET Webforms. There are many benefits of MVC over traditional asp.net .aspx web forms.

FEATURES OF MVC PATTERN

1. It is based on Model View Controller Pattern.
2. It is lightweight, fast and secure.
3. Designs are Codes are neatly separated.
4. It uses Pure HTML Control or HTML Helpers (No Server Control)
5. It uses controller based url.
6. Implement no view state.
7. It uses Razor Syntax for customization.
8. It uses Layout at the place of Master page for consistent looks and feel.
9. ASP.NET MVC is open source.

What's the "Core" part?

You have probably already noticed that this tutorial is called the "ASP.NET MVC **Core** Tutorial", but why Core? The original .NET framework, along with ASP.NET components, was originally released as a closed-source framework in 2002. Later on, Microsoft decided that they wanted to create an open-source .NET framework with support for the major operating systems: Windows, OS X and Linux. They called this the ".NET Core framework" and it was released in 2016, followed by many rapid releases with lots of improvements.

Today, the .NET Core framework is just as stable as the original .NET framework and besides compatibility with more operating systems it also comes with much better performance and a faster release cycle, meaning that you will get access to bug fixes and new features faster than with the original .NET framework. So, unless you need legacy functionality found only in the original .NET framework, **you should always go with the .NET Core version**